AI 超级智能体项目简历写法

编程导航原创项目教程：https://www.codefather.cn

作者：程序员鱼皮

建议

注意，以下简历写法仅供参考，根据自己的简历丰富度、以及对于项目的理解情况有选择地去写。**如果你自己还没有实现项目或者不理解，建议赶紧跟着鱼皮的教程把它弄懂，再写到简历上！**

本项目教程的每一节讲解的 AI 知识都是不同的，可以单独学习，如果时间紧迫，每学完一节都可以将部分内容写到简历上了。

此外，本项目以 AI 为核心，是很新的一类技术，因此完全可以把学到的知识点运用到你做的其他项目中，把该项目的亮点和你之前的项目进行整合。比如：

- 使用 Spring AI 开发框架
- AI 大模型接入和本地部署
- RAG 知识库构建
- 向量数据库应用
- AI 工具调用
- AI MCP 服务接入和开发
- AI 智能体构建
- Serverless 部署

可以使用鱼皮团队开发的 写简历神器《老鱼简历》 快速完整一份简洁专业的简历，有很多现成的简历模板：

专业技能

后端

可以写的内容非常多，大家挑选 4 ~ 8 点即可，千万别全部写上去了！

1. 熟悉 Java 21 特性，并能运用 Hutool、Lombok、Knife4j 等工具库提高开发效率，遵循阿里 Java 规范保证项目质量。
2. 熟悉 Java 主流类库的使用，如 Kryo 高性能序列化、Jsoup 网页抓取、iText PDF 生成、Tika 文档解析等，提高项目的开发效率。
3. 熟练使用 Spring Boot 3.x 框架独立开发后端项目，能整合 Knife4j 等组件实现接口文档自动生成。
4. 掌握多种 AI 应用开发方式，包括 SDK / HTTP / AI 开发框架 / 第三方大模型平台（如百炼和 Dify）等，能根据业务场景选择最优方案。
5. 熟悉 Spring AI、Spring AI Alibaba、LangChain4j 等主流 AI 开发框架，独立开发过企业级 AI 应用。
6. 掌握主流 AI 概念如 RAG、Tool Calling、MCP、CoT、ReAct、A2A 等，独立开发过拥有自主规划能力的 AI 智能体。
7. 掌握 Spring AI 框架，实践过结构化输出、多模态、ChatMemory、Advisor、RAG、Tool Calling、MCP 并通过阅读源码理解了原理。
8. 掌握 Spring AI 框架的核心特性，能快速实现具有会话记忆 + RAG 知识库 + 工具调用 + MCP 能力的 AI 智能体。

9. 掌握 Prompt 工程和优化技巧，比如思维链、少样本学习、多视角分析等，能够设计高质量的提示词并处理结构化输出。
10. 掌握 Spring AI 框架 RAG 应用开发，能通过自主实现文档加载器、分词器、高级文档检索策略来优化 RAG 效果。
11. 掌握 RAG 知识库构建技术，能基于 Spring AI 或百炼平台实现文档收集、向量转换与存储、文档检索和查询增强的完整流程。
12. 熟悉 PGvector 等向量数据库的应用，能结合 Spring AI 的 VectorStore 实现 AI 知识库管理和 RAG 智能检索。
13. 掌握 AI 工具调用 Tool Calling 特性，能利用 Spring AI 开发各种工具，比如网页抓取、联网搜索、PDF 生成等。
14. 掌握 MCP 模型上下文协议，能够使用、开发和部署 MCP 服务，熟悉 MCP 安全问题和最佳实践。
15. 熟练使用 Ollama 进行大模型本地部署和调用，理解大模型本地化的优势与限制。
16. 掌握 AI 智能体相关概念如 Cot 和 ReAct，深入阅读过 OpenManus 源码，并基于分层架构实现了拥有自主规划能力的 AI 智能体。
17. 掌握 SSE 技术，能运用 SSEEmitter 实现实时流式响应的 AI 对话服务接口，提升用户体验。
18. 熟悉响应式编程，能使用 WebFlux 实现非阻塞式异步操作，提高系统并发处理能力。
19. 熟悉 Docker 容器化技术，能够编写 Dockerfile 将应用打包成镜像，确保环境一致性和部署效率。
20. 熟悉 Serverless 部署技术，能将前后端项目快速部署到云平台，实现按需扩缩容和降低运维成本。
21. 熟悉主流设计模式，实践过单例模式、模板方法模式、代理模式、策略模式、工厂模式等并能快速阅读源码。

前端

这个项目的侧重点不在前端，以下仅为示例写法，不代表在项目中有过实践。

1. 熟悉 Vue 3、React 等主流前端框架，能自定义企业级前端项目模板，包括全局状态管理、用户登录态管理、动态权限控制、菜单生成与布局切换等功能。
2. 熟悉 Ant Design、Material Design 等 UI 库，能够利用 Ant Design X 业务组件库快速构建 AI 应用。
3. 熟悉前端工程化，能使用 Vite 构建工具实现前端项目打包和优化，结合 TypeScript 实现类型安全开发。
4. 熟悉前端代码规范，能使用 ESLint + Prettier + TypeScript 进行代码风格统一和质量检测，提升项目质量。
5. 熟悉 HTTP 协议和计算机网络模型，并且能够通过 SSE 技术实现对流式接口的调用和实时展示。
6. 熟悉 Axios 请求库，能自定义 Axios 实例，封装全局请求与响应拦截器，实现统一的错误提示并根据环境切换请求域名。

7. 熟悉前端性能优化，如 CDN 加速、懒加载、图片缩略与浏览器缓存策略，提升页面加载速度并节省流量。
8. 熟悉前端代码生成工具，比如利用 OpenAPI 自动生成 API 调用代码，提升开发效率并减少错误。
9. 掌握前端工具链，包括脚手架、Vite、Cursor、VS Code、WebStorm、pnpm、Git 等，能够快速搭建与管理前端项目。
10. 掌握前端项目部署，能运用 Docker + Serverless + Nginx 快速完成前端项目部署，并通过反向代理解决跨域问题。
11. 能利用 Cursor 等 AI 编程工具高效开发前端界面，通过 Prompt 工程快速生成符合需求的代码。

项目经历

项目名称：XX AI 超级智能体（比如 “鱼皮 AI 超级智能体”）

建议根据自己对项目的学习理解程度，自己想个有区分度的名字，其他名称参考：

- XX AI 恋爱大师
- XX AI 超级助手
- XX AI 工具箱
- XX AI 知识助手
- XX AI Agent

教程中，鱼皮只是以 AI 恋爱大师应用为例，带大家学习 AI 应用的开发场景和方法，大家其实可以尽情发挥想象力，开发属于你自己的 AI 应用，比如：

- AI 编程助手
- AI 智能客服
- AI 旅游规划大师
- AI 写作王
- AI 博客助手
- AI 自媒体文案助手

这样一来，就很容易拉开和其他同学简历的区分度。

GitHub 代码地址：https://github.com/liyupi/yu-ai-agent

建议大家也把项目放到代码仓库中，并且在主页文档里补充项目架构图、项目功能模块、技术选型等介绍信息。

但是注意不要抄袭鱼皮的写法！一定要加上自己的理解和扩展！ 否则一下就被面试官查出来了。

项目介绍

后端写法

可以根据简历的篇幅对下面的介绍进行适当的删减，一般项目介绍不要超过 3 行

基于 Spring Boot 3 + Spring AI + RAG + Tool Calling + MCP 的企业级 AI 恋爱大师智能体，为用户提供情感指导服务。支持多轮对话、记忆持久化、RAG 知识库检索等能力，并且基于 ReAct 模式，能够自主思考并调用工具来完成复杂任务，比如利用网页搜索、资源下载和 PDF 生成工具制定完整的约会计划并生成文档。

前端写法

基于 Vue3 + TypeScript 的企业级 AI 恋爱大师智能体，为用户提供情感指导服务。提供类似 ChatGPT 的响应式交互界面，支持多轮对话、RAG 知识库问答、工具调用和 MCP，基于 SSE 技术实时展示智能体的思考和执行过程。

主要工作

根据自己对项目的掌握程度，**选 6 ~ 8 个左右** 去写并适当调整文案，灵活一点。**强烈建议参考鱼皮的项目扩展思路多完善下项目，增加一些区分度！**

后端写法

冒号前的内容只是为了帮助大家快速理解每一点的核心内容，写在简历上时可以移除掉。

1. 项目架构搭建：基于 Spring Boot 3 搭建项目核心架构，整合 Hutool、Lombok 和 Knife4j 接口文档，并通过全局异常处理器增强了项目的健壮性。
2. 【推荐】AI 大模型集成：利用 Spring AI 框架快速接入通义、Ollama 等多种 AI 大模型，并封装统一的调用接口，实现了大模型的灵活切换，适应不同业务需求。
3. 本地大模型部署：基于 Ollama 实现了 AI 大模型的本地部署，用于处理简单的对话，解决了网络延迟和数据安全问题，同时降低了 API 调用成本。
4. 【推荐】Prompt 工程优化：运用角色定义、Few-shot 等技巧优化 AI 恋爱大师应用的提示词，并通过阿里云百炼平台反复测试，提高了模型回答的准确性。
5. Prompt 模板管理：基于 Spring AI 的 Prompt Template 特性实现可变参数的提示词管理系统，支持动态插入变量，避免了提示词硬编码，提高了提示词的灵活性和可维护性。
6. 【推荐】AI 多轮对话：通过 Spring AI 的 MessageChatMemoryAdvisor 和 ChatMemory 特性实现对话上下文记忆功能，使 AI 能在多轮对话中保持语境连贯。

7. 【推荐】对话记忆持久化：自主实现了基于文件系统的 ChatMemory，结合 Kryo 高性能序列化库保存对话历史数据，解决了服务重启后对话记忆丢失的问题，增强了系统稳定性。
8. 日志 Advisor 开发：通过实现 CallAroundAdvisor 接口开发日志 Advisor，记录 AI 对话请求和响应信息，便于开发调试。
9. 【推荐】Re-Reading Advisor 开发：通过实现 CallAroundAdvisor 接口开发 Re-Reading Advisor，通过让模型重复阅读用户输入，提高了复杂问题处理的准确性。
10. 【推荐】结构化输出：基于 Spring AI 的结构化输出特性实现了恋爱报告生成功能，将 AI 的回复精准转换为结构化 Java 对象，便于后续的数据处理和展示。
11. AI 知识问答：利用 QuestionAnswerAdvisor 实现了 RAG 检索增强生成，使 AI 能够基于特定知识库精准回答用户问题并推荐相关课程，提高了回答质量并增加了变现机会。
12. 【推荐】RAG 文档处理：使用 MarkdownDocumentReader 对恋爱知识文档进行读取和切片处理，并通过 KeywordMetadataEnricher 自动提取文档关键元信息以支持精确检索。
13. 【推荐】RAG 向量存储：自定义 VectorStore 实现，结合 EmbeddingModel 将文本转换为语义向量并存储到 PGvector 向量数据库，支持语义相似度搜索和多维度过滤，提高了检索精准度。
14. 【推荐】RAG 文档检索：基于 RetrievalAugmentationAdvisor 组合了文档检索器和查询转换器，通过配置相似度阈值和元信息过滤策略，优化了 AI 回复的准确性与相关性。
15. 【推荐】RAG 查询增强：综合运用多查询扩展、查询重写和上下文查询增强机制，优化用户原始 Prompt，提升了知识检索的召回率和 AI 回复的准确度。
16. 【推荐】RAG 云知识库集成：对接阿里云百炼知识库服务，通过 DashScopeDocumentRetriever 实现云端文档检索，简化了知识库维护成本。
17. ETL 数据处理：基于 Spring AI 框架实现了对恋爱知识文档的完整 ETL 数据处理流程，使用 DocumentReader、DocumentTransformer 和 DocumentWriter 实现知识文档的抽取、转换和加载，提高了知识库构建效率。
18. 文档质量优化：利用 AI 对原始知识文档进行内容结构化和格式标准化，使文档切分更加合理，提升了 RAG 的回答质量。
19. 批处理优化策略：设计并实现批处理策略 TokenCountBatchingStrategy，优化大量文档的向量化过程，解决了嵌入模型的上下文窗口限制问题，同时减轻了数据库压力。
20. 自动元信息标注：自定义实现 MarkdownDocumentReader 的配置，为文档自动添加元信息标注，比如文件名和恋爱状态标签，提升了多维度检索能力。
21. AI 关键词提取：利用 KeywordMetadataEnricher 实现基于 AI 的智能关键词提取和元信息增强，丰富了文档的元数据，提升了多维度检索能力。
22. 自定义文档切片：基于 TokenTextSplitter 自定义文档切片规则，通过配置最优分块大小和重叠比例，确保语义完整性和检索精准度。
23. 元数据过滤：基于 FilterExpressionBuilder 构造元数据过滤规则，实现了精确的文档筛选逻辑，使查询结果更加精准。

24. 多查询扩展：基于 MultiQueryExpander 实现智能查询扩展，将单一查询扩展为多个语义相关的变体，提高了检索的召回率和相关性。
25. 查询重写优化：基于 RewriteQueryTransformer 实现查询重写，将用户输入的原始查询智能优化为更清晰专业的表达，提升了检索的准确度。
26. 动态 Advisor 工厂：使用工厂模式，根据用户查询动态生成合适的 RAG Advisor，实现了查询与知识库的精准匹配。
27. 上下文查询增强：集成 ContextualQueryAugmenter 上下文查询增强器，通过自定义空上下文处理逻辑，当用户提出和恋爱知识无关的问题时，给出联系人工客服的友好引导。
28. RAG 参数优化：基于 DocumentRetriever 优化 RAG 检索参数配置，包括相似度阈值、返回文档数量和元信息过滤规则，提升了查询准确度。
29. 模块化 RAG 架构：基于 RetrievalAugmentationAdvisor 实现了模块化 RAG 架构，划分预检索、检索和后检索阶段，提高了系统的可扩展性。
30. 【推荐】AI 工具调用：利用 Spring AI 的工具调用注解实现了多种工具调用功能，包括文件操作、联网搜索、网页抓取、终端操作、资源下载和 PDF 生成，扩展了 AI 的能力边界。
31. 文件操作工具：封装统一的文件读写工具，并通过隔离文件存储目录实现了安全的文件系统访问，让 AI 能够保存用户恋爱记录文件。
32. 终端命令执行：通过 Java Process API 实现终端命令执行和结果获取，并针对不同操作系统进行了兼容性处理，让 AI 能够执行 Python 脚本来分析用户恋爱报告。
33. 资源下载工具：基于 Hutool 的 HTTP 工具类实现了指定 URL 的网络资源下载，让 AI 能够下载图片，使生成的恋爱报告更生动。
34. 网页抓取工具：基于 Jsoup 实现了网页抓取工具，能够提取特定 URL 的网页内容，为 AI 提供实时的外部信息。
35. 【推荐】联网搜索工具：基于 Search API 开发了网页搜索工具，支持从搜索引擎获取最新信息，增强了 AI 的实时信息获取能力。
36. PDF 生成工具：基于 iText 实现了 PDF 生成工具，支持中文内容和自定义排版，让 AI 能够为用户生成恋爱报告 PDF 文档。
37. 工具调用可观测性：通过配置 Spring AI 工具调用管理器的日志级别实现了工具调用过程的可视化，便于调试和排查问题。
38. 工具上下文传递：应用 ToolContext 上下文信息传递机制，在工具调用过程中携带用户身份信息，省去了用户需要自主告诉 AI 信息的麻烦，同时确保了多用户环境下的安全访问。
39. 【推荐】集中式工具注册：基于单例模式实现了集中式工具注册类，统一管理和配置所有可用工具。通过 Spring Bean 注入简化了工具的添加和移除流程，提高了代码的可维护性。
40. 【推荐】集成地图 MCP：利用 Spring AI 的 MCP Client 以本地 Stdio 模式集成高德地图 MCP，让 AI 能够准确地基于地理位置推荐约会地点。
41. 【推荐】图片搜索 MCP：利用 Spring AI MCP Server 集成 Pexels 图片 API 实现了图片搜索 MCP 服务，让 AI 能够联网检索图片资源；同时实现了 Stdio 和 SSE 两套传输模式，适应不同的部署场景。

42. 【推荐】部署发布 MCP：基于阿里云百炼平台以 Serverless 方式部署了图片搜索 MCP 服务，并将其发布到了 MCP.so 服务市场，供他人快速使用。
43. 【推荐】自主规划智能体：基于 ReAct 模式构建了具备自主规划能力的 AI 智能体，能够分解任务、自主决策、选择工具、循环执行直到完成复杂任务，比如帮用户自动寻找约会地点并生成约会计划 PDF。
44. 【推荐】分层智能体架构：参考 OpenManus 实现了拥有自主规划能力的分层 AI 智能体架构（BaseAgent、ReActAgent、ToolCallAgent），实现了高扩展性和可维护性。
45. 【推荐】深度思考：参考 OpenManus 的提示词实现了 CoT 深度推理模式，让智能体能够在处理复杂问题时逐步推理，显著提升了推理准确性和可解释性。
46. 【推荐】解决 Agent Loop 死循环：基于 Agent Loop 思想实现了 AI 智能体的自主多步骤执行，并通过最大步骤限制、Agent 状态管理和死循环检测，防止智能体陷入无限循环。
47. 智能体任务完成：设计实现了特殊工具机制，开发了终止工具让 AI 自主判断任务结束时机，有效解决了智能体执行流程控制问题，合理中断 Agent Loop。
48. 【推荐】AI 流式接口：使用 SseEmitter 实现了 SSE 流式响应 AI 应用接口，并通过 CompletableFuture 将 AI 推理任务异步化，避免阻塞主线程，提升了用户体验。
49. 推理过程实时输出：运用 SSE 实现了智能体执行进度的实时反馈，将原本黑盒式的 AI 推理过程转变为透明可见的步骤展示，提升了用户体验。
50. 跨域问题解决：掌握 CORS 跨域问题的解决方案，通过 WebMvcConfigurer 实现全局跨域配置，解决前后端分离开发中的跨域问题。
51. 【推荐】AI 服务部署：编写 Dockerfile 进行后端项目容器化，并通过云托管 Serverless 平台进行部署，能够自动扩缩容、降低运维成本并提高系统可用性。

前端写法

1. AI 前端项目生成：利用 AI 编程工具 Cursor 快速生成了基于 Vue3 的前端项目，实现了多页面切换、聊天室界面和实时消息显示等核心功能。
2. SSE 流式接口消费：设计实现了针对 SSE 流式接口的前端消费方案，实现了打字机效果的实时内容展示，大幅提升了用户体验。
3. 会话管理功能：开发了区分不同会话的聊天室功能，自动生成聊天室 ID 并维护会话状态，实现了多轮对话的隔离和连贯性。
4. 环境配置管理：实现前端环境配置管理，根据 process.env.NODE_ENV 环境变量自动切换 API 请求地址，简化了开发和部署流程。
5. 用户界面设计：设计开发了聊天界面 UI，区分用户消息和 AI 回复的展示样式，并优化响应式布局兼容多种屏幕尺寸。
6. 前端项目部署：编写 Dockerfile 进行前端项目容器化，集成 Nginx 实现静态资源服务和 API 反向代理，解决了跨域问题；并通过云托管 Serverless 平台进行部署，能够自动扩缩容、降低运维成本并提高系统可用性。

个人评价

1. 具备扎实的 AI 应用开发功底，熟练掌握 Spring AI、LangChain4j 等主流框架，能快速构建问答系统、自主规划智能体等 AI 应用。
2. 深入理解 RAG 检索增强技术，能从文档处理、向量存储到查询增强的全流程进行优化，提升知识问答系统的准确性和相关性。
3. 善于将复杂 AI 技术与实际业务场景结合，通过工具调用、MCP 服务集成等方式扩展 AI 能力边界，解决实际问题。
4. 具备 AI 系统架构设计能力，能从性能、安全、成本多角度进行技术选型和方案优化，如使用多级缓存提升响应速度、优化 Token 使用降低运营成本。
5. 熟练掌握 Prompt 工程技巧，能通过思维链、少样本学习等优化手段显著提升大模型输出质量，实现更精准的 AI 响应。
6. 有很强的自主学习能力，能通过阅读官方文档、开源项目和论文自学前沿 AI 技术，比如阅读 Spring AI 官方文档掌握了核心特性、参考 OpenManus 源码实现了类似的项目、通过国外 AI 资讯了解了 A2A 等新概念。
7. 有不错的产品思维，能够利用 AI 改进传统业务，比如能利用 RAG 知识库为用户推荐课程资源；能利用 SSE 技术实时展示 AI 的推理过程，提升用户体验。